Smart Medical Records Engine

Helps a doctor find the one thing in a patient file that matters, before the appointment ends.

A doctor asks in plain English and gets the answer out of that patient's own notes. The retrieval query carries the patient_id, so the database filters by patient before it ranks by similarity, which is what stops a different patient's note surfacing because it happens to be worded the same way. Thirty adversarial probes designed to leak cross-patient data, zero leaked, and the whole system runs on the hospital's own machine so no patient record leaves it.

Taiwo Alabi · MSc Artificial Intelligence, National College of Ireland

What this evidences: applying software architecture principles under a hard constraint. The model is the easy part. Scoping retrieval so the wrong patient cannot surface, and proving it, is the system.

Terms used in this report

Quick definitions before the technical detail starts, so nothing below assumes you already know these:

- **Embedding** - turning a patient's note into a row of numbers, so a computer can search by meaning instead of just matching words.
- **JWT** - a secure login token a doctor gets after signing in, checked on every request.
- **PII** - personal details like a name or address that could identify someone.
- **Retrieval** - the step of finding and pulling in the right piece of information before answering a question, rather than the AI guessing from memory.
- **Scoping** - restricting a search to one patient's own records before it even starts, rather than searching everyone and filtering afterwards.

Contents

1. The problem
2. What it does
3. How it works
4. The safety rail that matters most
5. Who is allowed in
6. The data model
7. Why self-hosted
8. Verified: thirty attempts to leak, zero leaks
9. Honest limitations
10. Running it

---

1. The problem

A doctor has ten minutes with you.

Somewhere in your file, which might be three years of notes from different visits, different doctors, different hospitals, there is a line saying you reacted badly to a particular medicine.

Your doctor is about to prescribe you something. Do they remember that line? Did they ever see it? Nobody reads three years of notes in ten minutes.

That is the whole problem. Not a modelling problem, a retrieval problem with a consequence attached.

---

2. What it does

A search box a doctor can talk to normally. They ask

"has this patient ever reacted badly to any medication?"

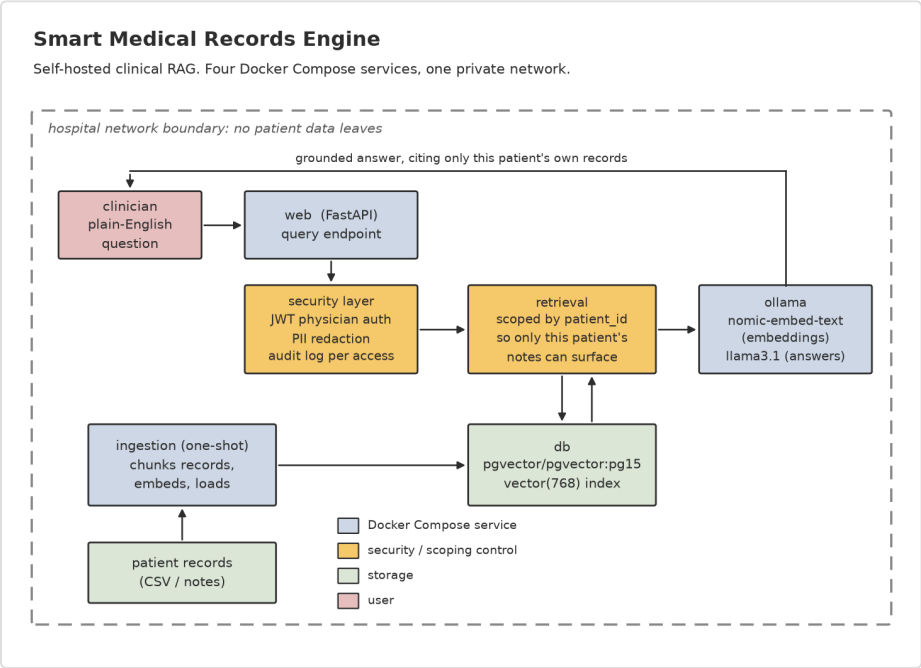
and it answers from that patient's own notes, showing the actual note it found the answer in.

Three things make it trustworthy, and all three matter more than the clever part:

- It only reads one file. Not a similar patient's, not someone with the same illness. Verified: section 8.
- Records never leave the hospital. The model runs on the hospital's own machine.
- Every access is written down. If anyone asks who opened a patient's file, there is an answer.

---

3. How it works



*Four services and a single hard rule: the retrieval query carries the patient_id from the request, and the database filters on it before ranking. Skip that step and a search will return a different patient's note because it happens to be worded the same way.**Four Docker Compose services on one private network. Orange marks the controls that constrain what the system is allowed to do.*

A question arrives at the service that handles requests. It passes the login check, gets turned into a row of numbers by `nomic-embed-text` (so it can be compared for meaning, not just matching words), and is used to search that one patient's notes in the database. The matching notes are handed to a locally-running AI model, `llama3.1`, as its source material, and the answer comes back quoting them directly.

Ingestion runs once as a separate service: it chunks the records, embeds them, and loads them with their patient reference attached.

---

4. The safety rail that matters most

Retrieval is scoped by patient before it ranks by similarity.

This sounds like a detail. It is the whole design. A general vector search over all notes would happily return another patient's record if that record happened to be the closer semantic match, and it often would, because two people with the same condition produce very similar notes.

So the filter is not a ranking preference applied afterwards. Wrong-patient results are excluded from the candidate set entirely, and the constraint is carried in the schema rather than left to application code.

---

5. Who is allowed in

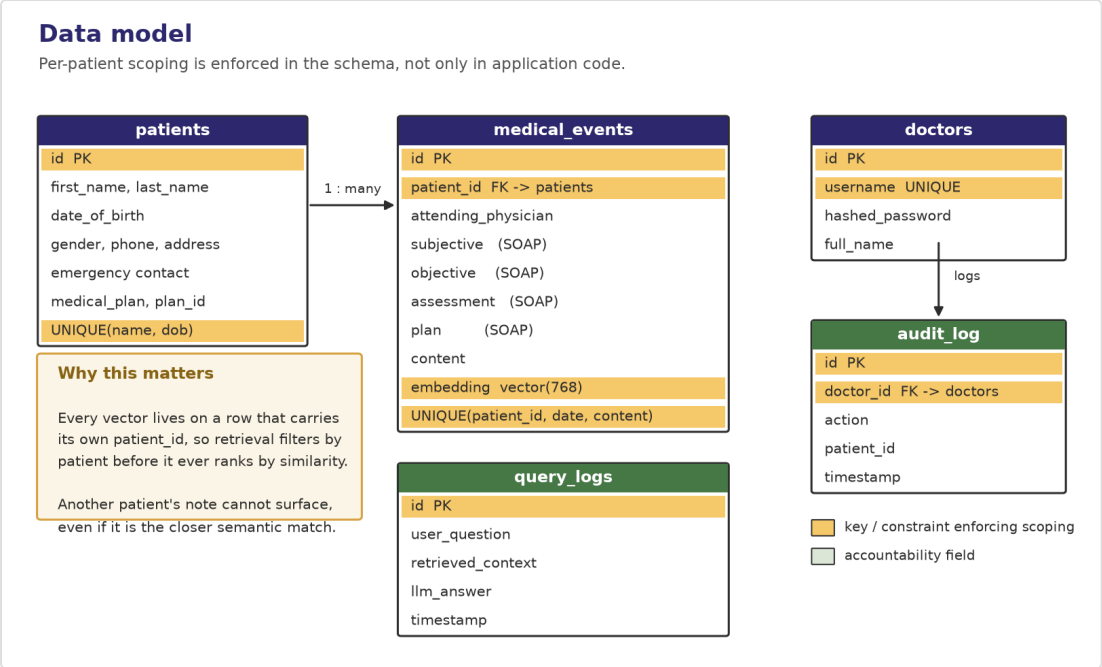
Doctors log in with JWT, a secure login token, and passwords are never stored in readable form, only a scrambled version that can check a password without keeping it.

Personal details, PII, like names and addresses, are stripped out before the text ever reaches the AI model, so it never sees identifying information it does not need in order to answer the question.

Every access writes an `audit_log` row against the doctor who made it. A system handling medical records that cannot answer "who looked at this, and when" is not deployable regardless of how well it retrieves.

---

6. The data model



The data model is the safety rail. Embeddings change when the model is updated. The patient_id column does not. Scoping happens at the database layer, not in application code, so the constraint cannot drift away from the search.

Notes are stored in SOAP format, subjective / objective / assessment / plan, because that is how clinicians already write them. Matching the existing format matters more than choosing a neater one: it means records can be ingested as they are.

Each note carries a `vector(768)` embedding on the same row as its `patient_id`, so the scoping constraint and the searchable content cannot drift apart.

---

7. Why self-hosted

Every model in this system runs locally through Ollama. No API calls, no data sent out.

That decision costs accuracy. A hosted frontier model would answer better. But patient records leaving a hospital network is not a trade-off a hospital can make, so the ceiling on model quality is fixed by the constraint, not the other way round.

The whole stack, API, database and language model, runs across four Docker Compose services on one machine.

---

8. Verified: thirty attempts to leak, zero leaks

The claim in section 4 is now tested rather than asserted. I made thirty attempts to trick the live application into leaking another patient's data, through the real path: signed in as a physician, asking questions designed to pull another patient's notes into the answer. Near-verbatim copies of a decoy patient's most distinctive lines. Semantic twins of the same clinical facts. Broad fishing questions. Queries naming the wrong patient outright.

Each attempt included a check to make sure it was a fair test: before counting it, I ran the same search without the patient filter, to confirm an unscoped search really would have surfaced another patient's note first. All thirty passed that check: in 30 out of 30 cases, the wrong patient's note would have been the top result. With scoping on, none of the thirty attempts leaked anything.

What this is not: a penetration test. No auth bypass was attempted, no injection, no infrastructure attacks. It verifies the scoping design through the application's own query path, which is the design this system stands or falls on.

Reproduce:

```
        python leak_probes/seed_fixture.py   # seeds a target patient + two near-duplicate decoys
       python leak_probes/run_probes.py     # runs the same thirty probes, prints a results table
```

---

9. Honest limitations

- It has not been tested with real clinicians. The retrieval works; whether the interface fits a ten-minute consultation is unproven, and that is the claim that matters.
- The data is synthetic. Real notes are messier: abbreviations, inconsistent formatting, scanned documents.
- A local `llama3.1` is weaker than a hosted model. Accepted deliberately, but it means answers on complex questions are less reliable than the retrieval underneath them.
- No formal evaluation set. Leakage was the priority, and it is measured in section 8: zero leaks across thirty attempts to trick it. Answer quality against a labelled benchmark remains unmeasured.

---

10. Running it

```
        docker compose up
       python scripts/create_doctor.py     # create a physician account
```

Ollama pulls `nomic-embed-text` and `llama3.1` on first run. Ingestion loads the records and builds the embeddings. Nothing reaches the internet after the model pull.

Stack: Python · FastAPI · PostgreSQL + pgvector · Ollama (llama3.1) · Docker Compose · JWT